

Sonido en videojuegos

FMOD Studio 2

El ejercicio marcado con **Evaluable** debe entregarse a través del CV. Para realizarlo:

- Crear un nuevo proyecto FMOD.
 - Incorporar los assets necesarios e implementar el evento pedido.
 - Comprimir el proyecto en un archivo **NombreApellidos1-NombreApellidos2.zip** (con los nombres de los alumnos del grupo). Solo subirá el archivo al CV uno de los miembros del grupo.
 - **NO** incluir el proyecto compilado! (la carpeta **Build** con los bancos generados).
 - Verificar que el proyecto entregado puede abrirse en el editor de FMOD.
-

1. (**Evaluable**) Utilizando las muestras de la carpeta tormenta vamos a crear un evento tormenta con varios parámetros para controlar la intensidad de la lluvia, la frecuencia de los truenos, el volumen global del evento y la posible obstrucción/oclusión del sonido. A continuación se detalla el funcionamiento de los parámetros (todos de tipo (*continuous*))

- *Intensidad*: utilizando distintas muestras de lluvia proporcionadas (pueden añadirse también otras disponibles en internet), este parámetro controla la intensidad del sonido de lluvia. Para ello puede integrarse las muestras en distintos tracks en un mismo evento y utilizarlas como capas sonoras controlando la mezcla de las mismas con el parámetro intensidad. Puede crearse el parámetro con una plantilla (*Parameter Sheet*), o crear el evento sobre el Timeline haciendo los ajustes apropiados.
- *Truenos*: utilizaremos distintos sonidos de truenos que se dispararán con mayor o menor frecuencia en función de este parámetro. Distintas muestras de truenos pueden sonar simultáneamente.
- *Volumen*: controla el volumen del master del evento.
- *Obstrucción y Oclusión*: estos parámetros servirán para simular la percepción del oyente desde un habitáculo (parcialmente) cerrado, como una casa o un coche. El efecto se consigue incorporando un *filtro paso bajo*. Para ello puede utilizarse el ecualizador multibanda con un filtro *Highshelf*. El parámetro obstrucción controlará la frecuencia de corte del filtro y la oclusión controlará la ganancia del efecto.

2. Crear un proyecto Unity vacío, incluir el plugin de FMOD y añadir el evento anterior. Crear un pequeño script para poder controlar cómodamente los parámetros del evento con el teclado y/o ratón.

3. El archivo *maquina.mp3* contiene una muestra de audio de máquina de escribir con varias pulsaciones y retorno de carro. Trocear la pista para obtener distintas pulsaciones individuales, muestras con varias pulsaciones y también una muestra con el retorno de carro. Después generar un evento con un multitímbrico que incorpore dichas muestras. Incluir moduladores de volumen y pitch con un pequeño rango de fluctuación, para crear variedad sonora. Para obtener un efecto realista deben asociarse probabilidades adecuadas a las pulsaciones y, en especial, el retorno de carro.

A continuación vamos a enrutar la señal y hacer un envío a un track de retorno con reverb para controlar la reverb en dicho track. Para ello

- Enrutar todos los tracks de audio anteriores a un nuevo track *Mezcla*
- Crear un nuevo track de retorno *Mi Reverb* y añadirle un efecto de reverb con **Wet Level=0** y **Dry Level=-∞**
- Desde *Mezcla* hacer un *Send a Mi reverb*

De este modo, la señal limpia debe sonar desde el track *Mezcla* y la reverberación se controlará con el Fader de *Mi Reverb*.

(Opcional) Implementar otro evento para conseguir un efecto más sofisticado utilizando un *Scatterer Instrument*, con *Polyphony=1* y controlando *Min & Max Spawn Interval* para incluir un intervalo de tiempo aleatorio entre pulsaciones.

(Opcional) Implementa un tercer evento teniendo en cuenta que una línea de texto en una máquina de escribir tiene un número limitado de caracteres (que obliga al retorno de carro), que al llegar al final de línea hay un pequeño timbre de aviso (una de las pulsaciones de la muestra incluye ese timbre), que puede haber líneas incompletas, varios retornos de carro seguidos. . . . ¿Se podría incorporar todo esto en un evento FMOD? En caso negativo, pueden implementarse eventos independientes para las pulsaciones, retorno de carro y timbre, y luego controlarlos desde la API con un autómata con transiciones probabilísticas entre estados.